

React

Day 1

jquery not much used now a days becoz of maintainance Instead Javascript is used all over.

→ Why js is written in body?

→ first a need to process UI & then perform js code based on UI hence written at end in body.

In MPA, we r shooting repetitive call to server & printing on screen.

But In SPA, only once call is shoot & then browser/js is creating itself a UI. Here size of js code ↑ heavily.

Angular gives SPA but at cost of heavy size/heavy js code Hence it's framework

→ Angular gives all thing inbuilt but react don't have inbuilt ex. for statefulness react don't give inbuilt support but we can get it with help of third party/library.

→ Using Webpack react compacts the file & sends the packet - minified version.

minifying is done automatically at package forming by react.

npmjs.com → Node package manager

repo where folder structure/packages are available.

some ex. redux - state management.

React → mostly used for SPA (single page application)

JSX → Javascript Extension. → JS + HTML.

Page No.	
Date	

react-bootstrap → for project

Router

↳ for navigation

Redux

state management

Axios

↓ Http client for browser

create react app create react app with no build config

* folder 19.3.0 → latest node.js

creation 8.19.3 → npm

① → Install Node from nodejs.org (install as side instal)

② → npm -v → To check version [v should be after 6]
node -v

③ → npm i create-react-app → local installation

④ → npx create-react-app firstdemo → extract the files
↑ & are added to firstdemo folder
in same folder where npm i cmd is run

Project name in react should in small case & no space.

npm start

- will compile ^{club} all JS file(s) into one file
add an entry into public/index.html
file about

<script src = "that one file created" > </script>

- also start react development server on some random port

- also open your default browser & also shoot a call to react development server based index.html

→ npm start will run only when a folder have node modules only i.e. if have dependencies

Never delete those files → package.json
package-lock.json

• gitignore → ignores the dependencies that we don't req i.e. we mentioned in package code.

3. Class component → class with some method generates VT
Functional component → function generates VT.

* Component:-

Home() is called as "component" in react.
here it is functional compo.

export → Unless we use export no one could able to access

import home from ~~./~~ "home"

→ Trying to create obj of home & it will further call render.
root.render(< Home / >); ← JSX syntax
Javascript HTML syntax

Code of react is executed on browser.

→ package.json → space & case sensitive.

All dependencies → present in Node Modules

* Class component:-

import { Component } from "react"

class About extends Component.

{ render()

→ method in component

↳ this method is called automatically using Reflection. render is coming from component.

reactjs.org
npm - installs permanently
npX - install only 1 time when
we give command.

Page No.	
Date	

bundle.js

First about obj is created & then
on that object render method is called

Simple react snippets → extension:

Js can't directly talk with html hence browser
creates some obj & we directly manipulate those
obj using document.getElementById.

→ here React does not directly talk with original
objs, it manages / manipulates replica of DOM

Original dom → div obj → UI side

→

~~Virtual Dom~~ → ~~connected~~ ~~is~~ ~~with~~ ~~cpu~~

Virtual DOM → take more memory. - ^{in.} memory

First changes done in virtual DOM & then in DOM.

→ package-lock.json → resolving dependencies
resolved path of dependency is stored here
- unless until an usage, it will not be touched.

* bundle.js → club multiple files together when
we give ~~bundle start~~ npm start
& done by react.

chrome → more → extensions → react developer tools

facebook, insta → react

imrc $\rightarrow$ import component.
cc $\rightarrow$ class

ex. ① return $\langle \underline{h_1} \rangle H_i \langle \underline{h_1} \rangle \langle \underline{b_1} \rangle \langle \underline{b_1} \rangle$ — not
allowed

2

→ allowed.
becoz div
is used as wrapper

③ return < >

→ allowed

using empty
tag.

 $\angle 1$

* String:-

State $\rightarrow$ JSON object stored as key-value pair

ex. state = { cname : "sunbeam" }

state.name ⇒ sunbeam
 ↑ ↓
 key value

`{ this.state.cname } → sunbeam` ^{so use} `set state.`
↳ this will change temporary & won't get reflected in UI

import Demo from './show'

↓

class name → file

{ this.do something }

↳ fun call

* ComponentDidUpdate: - after each time render happens becoz of set state
 - used for logging or updations to DB via XHR calls
 ↳ Xml Http Request

Page No	
Date	

* ComponentDidMount: - only once after first render
 - called once in whole lifecycle
 - used to instantiate var/obj.

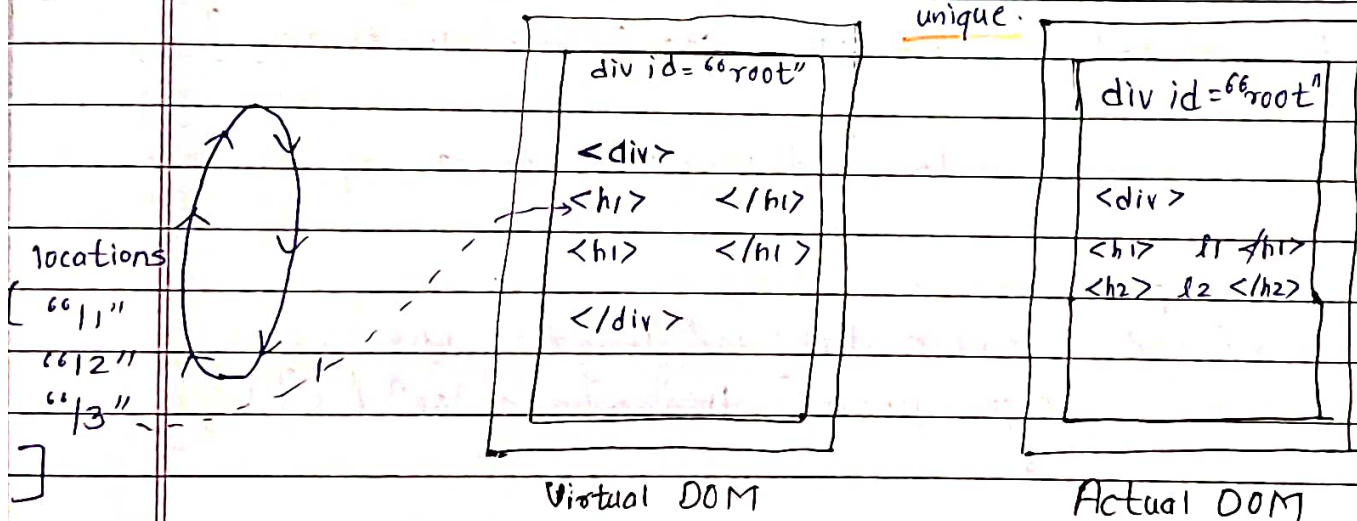
```
componentDidMount() {
  state.setState({ cname: "infosys" }) - initialize
```

* Array: -

→ In virtual DOM, to identify repetitive element react requires some property / id.

* Key attribute not in html but is in react.

<h1 key = { location } > { location } </h1>
 ↳ This key should be unique.



key should be any unique value

key is req for virtual DOM not actual DOM, becoz in case if a update value nxt time, to identify which 'loc' is changed virtual DOM will need Id (key attribute)

if home is class then create obj
 root.render(</Home>) → create obj of home & calls render internally.

return <branch> </branch> → makes object of branch & returns

Named Export

```

inline {
  export const name = 'J'
  export const age = 40
}

```

Default Export

export default message

Page No.

Date

export {name, age} - all at once at bottom

When branch is used in loop add key attribute.

<branch key = {location} > </Branch>

</Home> → if class then create obj & called render
 → if function, calls the fn

To add elements to existing array → arrayname.push()

→ let copyOfLocationsArray = [... this.state.locations

↑
①

copyOfLocationsArray.push("Nagpur").

ex. in orders.js

or ② this.setState = { orders: [... locations "nagpur"] }

this.setState({ locations: copyOfLocationsArray });

→ ... is used instead of loop to copy array

var copy

for(i=0; i < this.state.loc.length; i++)

{ copy.push(this.state.loc[i]); }

→ ... → destructuring syntax

① - exact replica of that object/array

② also can used as merger of array

ex. var nos = [1, 2, 3]

var names = [Po, Su, Si]

var all = ["hi", ...names, "bye", ...nos]

console.log(all) →

hi
 Po
 Su
 Si
 bye
 1
 2
 3

In case of array, on the value which we changed gets updated, other all values remains as it is. ex. firstdemo show.js

Page No	
Date	

In case of object, if only one field is set other is set to be null, ex. here address will be null.

→ Project name should be unique it matters when we upload it on npm.js. doesn't matter locally, matters when we publish.

→ Whenever we do setState → render → componentDidUpdate called always.

* Object:-

sample.js state = {
 person: { Name: "Nilesh", Address: "Pune" }
}
 key string object value.

→ In react it's compulsory to close the tag i.e.
 </br>

change = () => {
 this.setState(
 {

display both on UI ← person: { Name: "Amit", Address: "Pune" }
only display name ← person: { Name: "Nilesh" }
address will null

so to print new name & rest all attri. same case
make copy of that object & print that obj.

var copy = { ... person, Name: "abcd" };
console.log(copy); ⇒ abcd Pune → old address

Write only the property which you want to change.

* className → used to give name of class in CSS file.

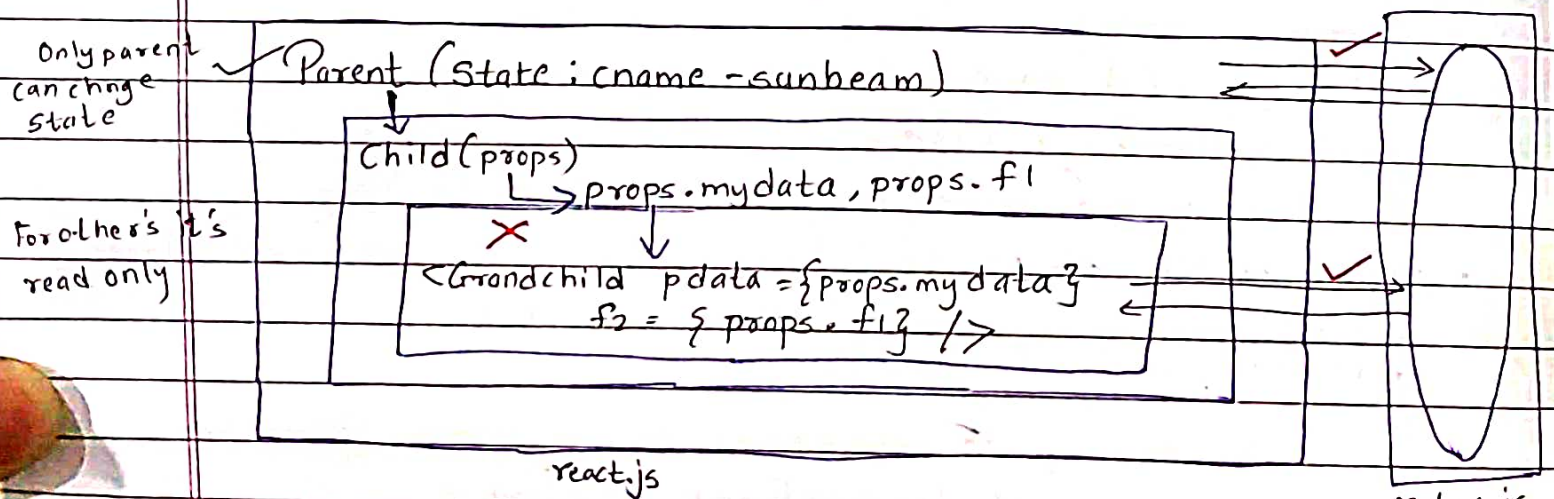
npm i bootstrap → latest version.

npm i bootstrap @ versioname → specified version

→ When we download bootstrap all dependencies gets install in node modules.

→ Props:- all time readonly.
 constructor(props) → this will make available props in ctor only
 {
 super(props) → prop is passed to parent ie component & then accessed in render & to any grandchild in read only state.
 }

→ One who create the state that comp. only can change the state in react



Props are read only hence solⁿ is to store state somewhere else, so that each individual can access data independently.

That external storage of data outside react is called as redux.js.

const root = ReactDOM.createRoot(document.

→ createRoot links virtual DOM with original. getElementById('root')

→ root contains ref on virtual DOM here we r catching ref of original DOM

In react, we are doing changes virtual DOM & it is reflected into original DOM.

ROOT node in react is Document.

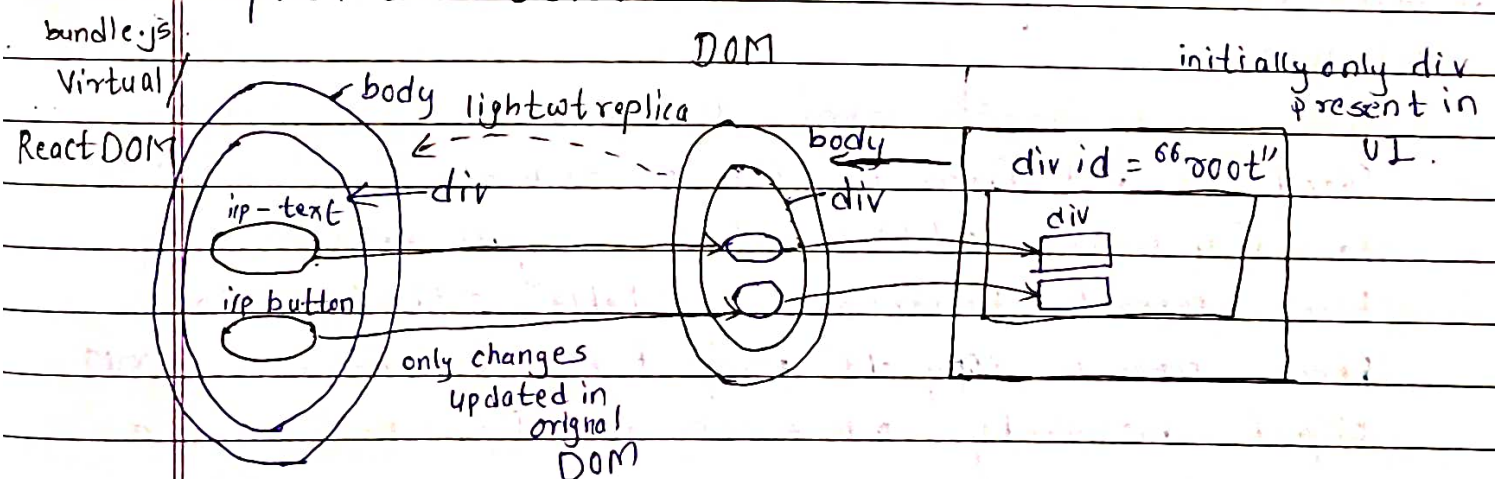
→ In react, Document is the parent object.

`createRoot()` → create a react root for supplied Id & return the root. This root can be used to render a react element into DOM with help of `render`.

Day 3

→ `npm init --yes` in `package.json`
 - gives some initial values
 ex. it gives `main = "index.js"` → default value
 so u can change ur starting file name here.

→ All .js files are bundled in `bundle.js` at start
`bundle.js` sees DOM & creates light weight replica of DOM.



lightwt replica of DOM is created by line `createRoot`

When ~~render~~ `root.render` is called all changes are added to virtual DOM. when `return` is executed actual DOM is updated & changes reflected to UI.

DOM mutation → event → done by browser.

Why we can't call `render()` explicitly?

`ShouldComponentUpdate` - becoz this method is by default present here. If we call `render()` explicitly then these methods are not called.

Parent object created.
(using constructor)

Lifecycle →

① state is initialized.

② component will mount if becoz of `setState`

→ `shouldComponentUpdate`

— returns true/false & calls `render` depending on True/False.

allows us to execute React code synchronously when comp. get loaded/mounted in

class Parent DOM
{
}

allows us to exit complex react update lifecycle to avoid it calling again & again. It only updates comp. if prop passed to it is changed.

③ `render`

if first time → `componentDidMount`

① customer

if becoz of `setState` → `componentDidUpdate`

② register

→ DOM nesting error: - when u don't write object hierarchy in proper sequence
→ React will not forgive u, it slaps error.

ex. if u write `<table> <tr>` it gives error

becoz correct hierarchy is `<table> <tbody> <tr>`

hence when u copy old project ensure correct DOM object hierarchy while copying DOM nesting is a concern.

To see keys used for uniqueness see it in component on F12

Constructor runs before anything else in class & it runs only once in entire lifecycle.

- When u want to initialize components before UI gets loaded, use ctor.

Page No.	
Date	

```
constructor(props)
{
  super(props)
}
```

↑
useless becoz i/p is as it is passed.

```
constructor(props)
{
  //manipulate props
  super(props)
}
```

useful coz props are changed

So if data is not modified it can be accessed directly as `this.props.data` → data is property name in parent.

In react we can't do `document.getElementById` becoz this works with actual DOM.

→ In i/p box we show by default value Sunbeam & now u want to change but it won't allow coz we can change state only by `setState` [i/p box is two way]

register.js So we call fun `onchange` → allow us to listen i/p change.

```
Handlechange
{
  console.log("typed")
}
```

Now this will notice our i/p but won't display change coz we r displaying state Sunbeam which is const.

Args → shows all obj's.

Args.target → shows i/p element → associated virtual DOM obj

Args.target.value → i/p value is shown (Sunbeam) here it's accepting value but not rendered hence shown still Sunbeam.

we can give any name to arg/arg1

args i.e. all obj are passed as arguments

Page No.	
Date	

Handlechange = (args) => { }

register

this.setState { v: args.target.value }

Now here new ip value is rendered by setState

state = { cname: "sun", address: "Pune" }

In plain html

to change something

① change in json.

person = { no: 1, name: "pune", address: "pooja" }

person.name = "Pooja"

② Square bracket / array person["name"] = "Pooja"

state = { customer: { cname: "pooja", address: "Pune" }

var copy = { ...this.state.customer, cname: args.target.value }

this.setState({ customer: copy });

* To reduce this repetitive code each time we give name to each ip tag.

var copy = { ...this.state.customer };

copy = { copy[args.target.name] = args.target.value; }

this.setState({ customer: copy });

name = cname / address

className = "btn btn-Danger" → Delete button
 className = "btn btn-primary" → blue coloured btn
 "btn btn-info" → light blue. ex. to add employee.

table table-bordered, table table-responsive
 ↳ bordered table. ↳ creates responsive table.

Table will scroll horizontally on small devices (< 768px)
 > 768px no effect.

here {} → for JavaX not.

∴ style = { { columnSpan: 2 } }

→ onClick = { this.remove(emp.No) }
 this will directly call emp no remove & all contents deleted

→ onClick = { () → { this.remove(emp.no) } }

→ here were passing emp.no to anonymous fuⁿ
 fd hence on that button click resp. record will deleted.

Filter function is similar to map fuⁿ.

var filtered = this.state.employees.filter((emp) ⇒
 { *return (emp.No != no) ; });

this.setState({ employees : filtered });

→ setState is asynchronous fuⁿ. it figure out all the changes in same fuⁿ & when fuⁿ scope ends it compares & only send the change.

ex. in add fuⁿ setState called twice but render is called once only & for 2nd setState.

To process something when u type an UI write onChange

* Search:-

if(emp.City.includes

* Functional Component:- this keyword is not used here

till 2018 transfer of data & event handling wasn't possible.
Initial usage of fuⁿ comp → only returns UI, only for presentⁿ logic.

After 2018, fuⁿ comp. got state, event handling with help of hook.

* useState fuⁿ / Hook:-

var [bookName] = useState("let us C");

here let us see is default value of bookName.

This isn't class comp.

bookName is accessed directly w/o this keyword
Name: {bookName}

→ No render fuⁿ in fuⁿal compo.

fuⁿ Book()

{ var [bookName] = useState("let us C");
return <div>

Hook

Name: {bookName}

}

</div>

<button onclick={change}>Click

</button>

* For String:-

```
var [BookName, setBookName] = useState("Let C");
```

↳ fun & declarⁿ in usestate
↳ given by usestate

```
var change = () →  
  { setBookName("Let not C"); }
```

→ instead of render only return all time instead of state, var is written.

* For Object:-

```
var [bookDetails, setBookDetails] = useState(  
  { bookName: "Let C",  
    bookAuthor: "kanetkar" } );
```

Name: { bookDetails.bookName }
 { bookDetails.bookAuthor }

→ setState → key value pair req.
setbookdetails → not req. ex setBookDetails(copy);

```
var copy = { ...bookDetails, bookName: "Not C" }
```

or

```
var copy = { ...bookDetails };  
copy.bookName = "Not C"
```

* For Array:- Array of books

```
var [books, setBooks] = useState([  
  { id: 1, bookName: "C", author: "A" },  
  { id: 2, bookName: "J", author: "B" }  
]);
```

```
books.map (book => {
  return <li key = { book.id } >
    { book.id + book.bookName }
  </li>
```

function

```
var Add = () =>
{
  var copy = [... books];
  copy.push ( { id = 17, bookName = "A" } );
  setBooks (copy);
}
```

useState → only for state management.

to understand lifecycle management → use effect.
↳ componentDidMount & componentDidUpdate

* useEffect Hook:-

```
useEffect ( () => {
```

```
  }, [bookName, author] )
```

↳ after changing a bookName or author or both, call useEffect.

In class, we were calling `com.DidUpdate` after every chge of bookName / author. i.e. here it will be called twice.

But in fun, after updating any of two or both useEffect is called.

→ map → only works with array, if any pt u converted array to object then single object can't be mapped

useEffect (() => { }, [bookName]) ①

useEffect (() => { }, [Author]) ②

useEffect (() => { }, [bookName, Author]) ③

If we chnge only bookName, both ① & ③ will be called.

→ var [author, setauthor] = useState(" ");

→ Whenever we initially setup value it still call ~~did mount~~ did update ie. useeffect is called initially also.

→ useEffect (() => { }, [])

useEffect with no parameter ie. blank array $\approx$ comp. Did Mount

called only once
ie. first time

→ UseEffect with blank array → component Did Mount

useEffect with any parameter → component Did Update.

String. includes (searchString : String , position? : (number | undefined)) : boolean.

→ Includes return true if searchString appears as result , at 1 or more posⁿ that are greater than or equal to posⁿ , else false.

parameters → searchString - string

position - if posⁿ undefined, 0 is

assumed so as to search all string

→ To change meaning of 'this' to current fuⁿ.

f1 = this. DoSomething. bind (this)

not recommended

write

DoSomething ()

arrow

fuⁿ instead.

{ console.log ("I clicked " + this.state.name);

}
render () { return (<button onclick = { this.f1 } </button>)

Day 4

→ In class comp, component will update called many times & whenever if there is change or not irrespective of state chge, component will update is called.
ie. if setState / state anyone written.
 before comp. will update, should comp is called.

→ Whereas in functional comp. only if there is change, then only use effect of resp. change is called. hence it's effect if no chnge no one is called.

→ In functional component, no need of key & only have to write value.

→ function Users (props)
 {
 - can pass/access prop like this
 }
 - meaning of prop is same in class & functional
 → one who write can only change & for others props are read only.

→ react-router-dom → for navigation.
 npm i react-router-dom.

* Routing / Navigation from 1 comp. to other:
 ↓ link is component just like our home/about.

→ link components → internally gets converted to a href.
 ex. <Link> Home </Link>

<Route path = "/" element = <Home />
 ↳ When user / comp. display Home comp.
cell

element = { < /Home > }

→ is for react DOM.

`<Route path = "*" component = { NotFound } />`

if user type anything that we have not specified show not found comp.

lly to default case in switch.

Route is ^{don't produce anything on UI} for react understanding, Link is for UI.



`<Link to = {"/home"}> Home </Link>`

`<Link to = {"/about"}> About </Link>`

Routes
~~<switch>~~
`<Route path = "/"`

version 5.2.0

`<Route path = "/login" component = {login} />`

~~component~~ = {</Home>} />
element

→ here we r inside a div, virtual DOM, but who will going to convey changes to browser? Browser Router
Browser Router → mediator betⁿ browser & react applⁿ, convey changes to Browser.

→ `root.render(
 <&u>BrowserRouter>
 <Main UI> </Main UI>
 </BrowserRouter>)` When we type in URL the changes should convey to react & it is done by browser route.
If user click URL/link, resp url gets changed in tab.

`<Route path = "/home" element = { </Home> }>`

if /home typed in URL / clicked goto Home component url

`<Link to = {"/ Home"}> Home </Link>`

→ If we type in URL /home or anything whole page is refreshed but if we clicked on any link inside UI it will display only changes no refresh occurs.

→ Page not found. we won't get when we click but if wrong URL is typed ex. /Home1 then we get page not found.

→ ^{Use History} ~~useNavigate~~ → refer browser History.
hook to access History from browser!
navigate history from A locⁿ to B locⁿ

→ useNavigate → hook instead of useHistory() in current
↳ react-router-dom. Version 6.6.2

```
var navigate = useNavigate();
var Navigate = () => {
  if (loggedIn) {
    navigate('/dashboard');
  } else {
    navigate('/login');
  }
}
```

* Factory class → returns a some object based on a logic always.

In factory class we write some logic & based on logic some object is returned.

In react, factory method is generally called Protected Route.

<Protected Route path = '/Dashboard'

main VI

element = { <Dashboard /> } />

here path & element is passed as props to protected route.

& checking condⁿ protected route returns either dashboard or login.

function ProtectedRoute (props)

{

var is-login = true.

if (true)

//check for session storage

return <Route path = { props.path }

element = { <props.element /> } />

}

login & crud is imp acc. to CCE, lab

→ protected root is wrapper around some components.
wrapper around login & dashboard.

npm i react-router-dom@5.2.0

History.push → push a path or location to history stack.

→ helper.open("POST", "http://^{URL} /auth");

↳ whatever we write here goes

in header of packet - if GET

helper.setHeaders({ key: value }) & body of packet - if POST

Q. When we shoot a call to web API, how to authorize?

→ There is token based system. Storing of token is done by browser.

Switch management in Web API concept. Hence our self has to do is token id sent along with set header from client to server? if yes by browser or by program?

Day 5

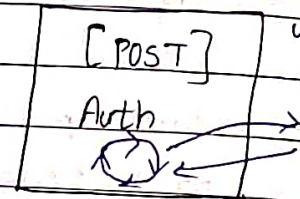
var helper = new XMLHttpRequest();

helper.onreadystatechange = () => {}

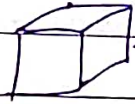
helper.open("POST", "http://.../auth");

helper.send({credentials});

compare with DB



valid token
browser storage



JSON format credentials

→ * In case of web API (stateless)

while authorising, 2 methods to remember user

① Browser stores in DB. every time request come it comp. with DB.

② First time req. come ^{generate & server} sends new ~~browser~~ ^{encrypts} the token then every time new req come ~~browser~~ ^{server} only checks the encrypted token each time.

storage by session/local

hence we have to explicitly do statefulness
Web API concept no session so we have to maintain token; ex. rest API using springboot we used token.

Web API → doesn't create VI

we just send req to it via → Get, Post, Put, Delete

Page No.			
Date			

(check headers on F12, Network area)

Proxy m/c are normally setup for network management.

https → https server always demand middle m/c's should be secured / safe.

i.e. only m/c who use https will only allowed to pass packet.

→ If packet is tampered packet will be destroyed at end.
feeder tool → network monitoring

How to assure packet is not tampered?

With help of 3rd party certificate authority.

If packet tampered, certificate is destroyed.
certificate authority → ensures no one can read body content only head.

preflight → ① When u r working with cross-browser.
along with our req / preflight req is also sent.

→ Https:-

ensures safe transport of data packet
we can only access - Header of packet & not body.
When req. is send to Http server, it will check whether data packet network - safe or not, if not packet destroy & send req. to from client

In case of ~~the~~ spring project → stateful.

wh api i.e. in case of java, .net we use concept of session
but Rest/web api → stateless.

in react we r doing web api hence statelessness

in web no concept of session coz of statelessness, tokens are used instead of session in web app / api

XHR calls → JQuery wrapper - Ajax
then → what will happen if resolve is called
React wrapper - Axios.
Fetch.

Page No.	
Date	

outsourcing responsibility to promise

If we're sure that there will success → only call then.
eg. promise.then(on success)

If not sure write catch. promise.catch(on failure)

then can be called any no. of times

→ promise.then(on success);
promise.then(() ⇒ { console.log("Hi") });
promise.catch(on failure);

chaining →

promise.then(on success).then(() ⇒ { console.log("Hi") })
• catch(on failure)

In react fetch fn is provided by react we just have to write then & catch

demo promise 2.html → fetch fn in whole js

by default call in fetch fn → GET

test promise.js → fetch fn in React & axios fn in React. } both are used for XHR calls

What is promise fn asked?

No one ask fetch/axios but b/c promise is asked

Axios → Outsourcing way of handling XHR calls
- used in Asynchronous method.

Promise is working behind both fetch & axios